Introduction

A language model reads text as a sequence of tokens, so the tokenizer decides how much text fits in the model's context and how much computation each word costs. The tokenizer of GPT-2 was trained mostly on English text, and it breaks Urdu words into small byte fragments. This assignment builds a tokenizer for Urdu from an empty folder, in three tasks: collecting and cleaning at least 500 MB of Urdu text (Part A), implementing byte-level byte-pair encoding (BPE) from scratch and training it to 50,000 tokens (Part B), and analysing what the tokenizer learned and comparing it with existing tokenizers (Part C). The corpus and the tokenizer are the inputs to the next assignment, where a GPT-style model is trained on them, so a weakness in either carries forward.

Problem

An Urdu letter takes two bytes in UTF-8, and most Urdu words are missing from the GPT-2 vocabulary, so they are split into many short pieces. Sequences become long and training becomes more costly. Two further problems are specific to Urdu. First, one word can be typed with different code points that look identical on screen (for example Arabic and Urdu yeh, or ZWNJ inserted by some keyboards), and a tokenizer learns every variant as a separate token unless the text is normalised first. Second, the usual pre-tokenisation pattern treats diacritics as punctuation and cuts words apart. The rest of this report follows the three tasks: building the corpus, building and training the tokenizer, and analysing the result.

Part A: Corpus

The final corpus is about 1.2 GB of cleaned Urdu text, well above the 500 MB required. About 300 MB was collected by us from Urdu news and blog sites, following robots.txt and limiting the request rate. The rest comes from the Urdu Wikipedia, Wikisource, Wikibooks and the CC-100 web corpus, so the corpus covers news, opinion, blog, encyclopedia, book and web text. Cleaning removed lines that are not Urdu, duplicate documents, repeated boilerplate such as menus and "read more" lines, and repeated paragraphs. The text was split into training, validation and test files (98%, 1% and 1%) by document, so no document appears in two splits. The data card lists every source, licence and cleaning step.

Part A: Normalisation

The same Urdu word is often typed with different code points that look identical. The normaliser (normalize.py) maps Arabic yeh, kaf and heh to their Urdu forms, removes tatweel and invisible characters, and makes spacing uniform. It runs before training and never inside the tokenizer, so the tokenizer can encode and decode any text exactly. Two choices needed a decision. ZWNJ is removed, because it is rare and used inconsistently: in most words that contain it, the same word without it is also common in the corpus, so keeping it would make the tokenizer learn two spellings of one word. The cost is that a few real word boundaries are joined. Digits are written as ASCII digits, as almost all digits in the corpus already are. Diacritics are kept, since they are part of Urdu text, especially in poetry and religious writing.

Part B: Pre-tokenisation pattern

BPE only merges inside the pieces produced before training, so the pre-tokenisation pattern defines what counts as a word. The GPT-2 pattern matches letters only. Urdu diacritics are combining marks, so GPT-2 treats them as punctuation and cuts a word such as اُردُو into five pieces. A zero-width non-joiner (ZWNJ) cuts a word in the same way. Our pattern keeps both inside the word. It is given below and its five alternatives are explained after it:

(?i:'s|'t|'re|'ve|'m|'ll|'d)| ?[\p{L}\p{M}\u200c\u200d]+| ?\p{N}+| ?[^\s\p{L}\p{M}\p{N}\u200c\u200d]+| \s+(?!\S)|\s+

1. English contractions stay together, since English words appear inside Urdu text.
2. A word is a run of letters, combining marks and the joiners ZWNJ and ZWJ, with an optional space before it. Diacritics and joiners therefore cannot split a word.
3. A number is a run of digits in any script.
4. Punctuation, symbols and emoji form separate pieces. Urdu punctuation is split from the word before it even when no space is written.
5. The last space before a word stays with that word, as in GPT-2, and other whitespace such as a newline is a separate piece.

As a result, no merge can cross a word, number or punctuation boundary.

Part B: Training and verification

The vocabulary has 256 byte tokens, the special token <|endoftext|> and 49,743 learned merges, which gives 50,000 tokens. When two pairs have the same count, the greater pair (compared as bytes) is merged, so training is deterministic. We implemented two trainers. The reference trainer recounts all pairs after every merge, which is simple but slow. The fast trainer counts each distinct word once, updates only the words affected by a merge, and caches the encoding of words it has already seen.

Both trainers learned identical merges on the standard example (aaabdaaabac) and on a 1 MB sample of the corpus (first 1,000 merges). On this sample the reference trainer took about 230 seconds and the fast trainer 1.5 seconds. Training the final tokenizer on the full training file took 288 seconds (about 5 minutes) of wall-clock time on a laptop with an Intel Core i7-8665U processor (8 logical cores) and 15.7 GB of RAM, without a GPU.